

Assignment 3

Analyze the two GenAI programs. The analysis must include:

1. Name of two GenAI programs

Answer: Gemini and Claude

2. How you accessed them (e.g., Google query, CoPilot on my computer)

Answer: Accessed them with their official websites:

Gemini: <https://gemini.google.com/>

Claude: <https://claude.ai/>

3. Prompt used. (You must use the same prompt for both GenAIs.)

Answer:

Objective:

Create a C++ program using objects, not functions, that implements an email priority system for a busy CEO. The program must use a MaxHeap as a priority queue to handle the emails based on priority. The MaxHeap must use a list-based implementation, and then use that MaxHeap to handle all email prioritizing for the CEO. The MaxHeap must be implemented from scratch. DO NOT USE pre-existing heap modules.

Requirements:

1. The program must read commands from a test file.
2. The supported commands are:
 - EMAIL <sender category>,<subject line>,<date>
 - NEXT
 - READ
 - COUNT

3. Each email contains:
 - Sender category
 - Subject line
 - Date in MM-DD-YYYY format
4. Sender priority from high to low must be:
 - Boss – read first
 - Subordinate – read next
 - Peer – read next
 - ImportantPerson – read next
 - OtherPerson – read last
5. If two emails have the same sender category, the newest email must have higher priority.
6. NEXT must display the highest-priority unread email without removing it. Display the information on the terminal in the following format:
 - Sender:
 - Subject:
 - Date:
7. READ means the CEO has read the email and dealt with it.
8. COUNT will display the current count of unread emails.
9. Two NEXT commands in a row without a READ must display the same email both times.
10. Two READ commands in a row must remove the two highest-priority emails, even if NEXT was not called first.
11. The program must safely handle NEXT or READ when there are no emails.
12. The subject line may contain spaces but will not contain commas.
13. Use appropriate C++ objects/classes instead of a function-based design to keep the program organized and maintainable.
14. Implement the MaxHeap operations from scratch using objects/classes, including heap insertion, heapify-up, heapify-down, comparison, peek, and removal operations.
15. Include clear comments explaining the major parts of the code.

4. Generated code from both GenAIs.

Answer:

Gemini Code:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <string>
```

```
#include <vector>
```

```
// Enumeration for Sender Priority (lower numerical value = higher priority)
```

```
enum class SenderCategory {
```

```
    Boss = 1,
```

```
    Subordinate = 2,
```

```
    Peer = 3,
```

```
    ImportantPerson = 4,
```

```
    OtherPerson = 5,
```

```
    Unknown = 6
```

```
};
```

```
// Represents an individual email
```

```
class Email {
```

private:

std::string senderStr;

SenderCategory category;

std::string subject;

std::string dateStr;

int year;

int month;

int day;

// Helper to map string category to Enum

SenderCategory parseCategory(const std::string& cat) {

if (cat == "Boss") return SenderCategory::Boss;

if (cat == "Subordinate") return SenderCategory::Subordinate;

if (cat == "Peer") return SenderCategory::Peer;

if (cat == "ImportantPerson") return SenderCategory::ImportantPerson;

if (cat == "OtherPerson") return SenderCategory::OtherPerson;

return SenderCategory::Unknown;

}

// Helper to parse MM-DD-YYYY for date comparison

void parseDate(const std::string& d) {

std::stringstream ss(d);

```
std::string m, dayStr, y;

std::getline(ss, m, '-');

std::getline(ss, dayStr, '-');

std::getline(ss, y, '-');

month = std::stoi(m);

day = std::stoi(dayStr);

year = std::stoi(y);

}
```

public:

```
Email(std::string sender, std::string subj, std::string date)

: senderStr(sender), subject(subj), dateStr(date) {

category = parseCategory(sender);

parseDate(date);

}
```

```
std::string getSender() const { return senderStr; }
```

```
std::string getSubject() const { return subject; }
```

```
std::string getDate() const { return dateStr; }
```

```
// Priority Comparison logic:
```

```
// Returns true if 'this' email has HIGHER priority than 'other'
```

```

bool isHigherPriorityThan(const Email& other) const {
    if (this->category != other.category) {
        // Lower numeric enum value means higher priority (e.g. Boss=1 <
        Subordinate=2)
        return this->category < other.category;
    }

    // If categories match, compare dates (Newest email wins)
    if (this->year != other.year) return this->year > other.year;
    if (this->month != other.month) return this->month > other.month;
    return this->day > other.day;
}
};

```

// Node structure for List-Based Binary Heap implementation

```

struct HeapNode {
    Email data;
    HeapNode* parent;
    HeapNode* left;
    HeapNode* right;

```

```

    HeapNode(Email email)

```

```
        : data(email), parent(nullptr), left(nullptr), right(nullptr) {}  
};
```

```
// Custom List-Based (Tree-based) MaxHeap Class
```

```
class MaxHeap {
```

```
private:
```

```
    HeapNode* root;
```

```
    int size;
```

```
    // Helper function to find a node by its 1-based index (Level-order  
    Traversal via Bit Path)
```

```
    HeapNode* getNodeAtIndex(int index) const {
```

```
        if (index <= 0 || index > size) return nullptr;
```

```
        if (index == 1) return root;
```

```
    // Collect path bits from root to target index
```

```
    std::vector<int> path;
```

```
    while (index > 1) {
```

```
        path.push_back(index % 2); // 0 = left child, 1 = right child
```

```
        index /= 2;
```

```
    }
```

```

HeapNode* curr = root;
for (int i = path.size() - 1; i >= 0; --i) {
    if (path[i] == 0) curr = curr->left;
    else curr = curr->right;
}
return curr;
}

```

// Swaps email payloads between two nodes

```

void swapData(HeapNode* a, HeapNode* b) {
    Email temp = a->data;
    a->data = b->data;
    b->data = temp;
}

```

// Restores heap order upward starting from the given node

```

void heapifyUp(HeapNode* node) {
    while (node->parent != nullptr && node-
>data.isHigherPriorityThan(node->parent->data)) {
        swapData(node, node->parent);
        node = node->parent;
    }
}

```



```
}
```

```
// Restores heap order downward starting from the given node
```

```
void heapifyDown(HeapNode* node) {
```

```
    while (node->left != nullptr) {
```

```
        HeapNode* maxChild = node->left;
```

```
        if (node->right != nullptr && node->right->data.isHigherPriorityThan(node->left->data)) {
```

```
            maxChild = node->right;
```

```
        }
```

```
        if (maxChild->data.isHigherPriorityThan(node->data)) {
```

```
            swapData(node, maxChild);
```

```
            node = maxChild;
```

```
        } else {
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
// Dynamic clean-up for deep tree deletion
```

```
void clear(HeapNode* node) {  
    if (!node) return;  
    clear(node->left);  
    clear(node->right);  
    delete node;  
}
```

public:

```
MaxHeap() : root(nullptr), size(0) {}
```

```
~MaxHeap() {  
    clear(root);  
}
```

```
int getSize() const {  
    return size;  
}
```

```
bool isEmpty() const {  
    return size == 0;  
}
```

// Insert a new Email into the heap

```
void insert(const Email& email) {  
    HeapNode* newNode = new HeapNode(email);  
    size++;  
  
    if (root == nullptr) {  
        root = newNode;  
        return;  
    }  
  
    // Find the parent node where the new child must attach  
    HeapNode* parentNode = getNodeAtIndex(size / 2);  
    newNode->parent = parentNode;  
  
    if (size % 2 == 0) {  
        parentNode->left = newNode;  
    } else {  
        parentNode->right = newNode;  
    }  
  
    heapifyUp(newNode);  
}
```

// Peek at the highest priority Email without removing it

```
bool peek(Email& result) const {  
    if (isEmpty()) return false;  
    result = root->data;  
    return true;  
}
```

// Remove and return the highest priority Email

```
bool extractMax(Email& result) {  
    if (isEmpty()) return false;  
  
    result = root->data;  
  
    if (size == 1) {  
        delete root;  
        root = nullptr;  
        size = 0;  
        return true;  
    }
```

// Get last node in complete binary tree

```
HeapNode* lastNode = getNodeAtIndex(size);
```

```
HeapNode* lastParent = lastNode->parent;
```

```
// Move last node's content to root
```

```
root->data = lastNode->data;
```

```
// Disconnect last node
```

```
if (lastParent->left == lastNode) {
```

```
    lastParent->left = nullptr;
```

```
} else {
```

```
    lastParent->right = nullptr;
```

```
}
```

```
delete lastNode;
```

```
size--;
```

```
heapifyDown(root);
```

```
return true;
```

```
}
```

```
};
```

```
// Managing Class to control CEO Inbox workflow
```

```
class EmailInboxManager {  
private:  
    MaxHeap priorityQueue;  
  
public:  
    void addEmail(const std::string& line) {  
        std::stringstream ss(line);  
        std::string sender, subject, date;  
  
        if (std::getline(ss, sender, ',') &&  
            std::getline(ss, subject, ',') &&  
            std::getline(ss, date)) {  
  
            Email email(sender, subject, date);  
            priorityQueue.insert(email);  
        }  
    }  
  
    void displayNext() {  
        Email email("", "", "");  
        if (priorityQueue.peek(email)) {  
            std::cout << "Sender: " << email.getSender() << "\n";  
        }  
    }  
};
```

```

        std::cout << "Subject: " << email.getSubject() << "\n";

        std::cout << "Date: " << email.getDate() << "\n";

    } else {

        std::cout << "No unread emails.\n";

    }

}

```

```

void readNext() {

```

```

    Email email("", "", "");

```

```

    if (!priorityQueue.extractMax(email)) {

```

```

        std::cout << "No unread emails to read.\n";

```

```

    }

```

```

}

```

```

void displayCount() const {

```

```

    std::cout << "Unread emails count: " << priorityQueue.getSize() <<
"\n";

```

```

}

```

```

// Command File Processor

```

```

void processCommandFile(const std::string& filename) {

```

```

    std::ifstream file(filename);

```

```
if (!file.is_open()) {  
    std::cerr << "Error: Could not open file " << filename << std::endl;  
    return;  
}
```

```
std::string line;  
while (std::getline(file, line)) {  
    if (line.empty()) continue;  
  
    if (line.rfind("EMAIL ", 0) == 0) {  
        addEmail(line.substr(6));  
    } else if (line == "NEXT") {  
        displayNext();  
    } else if (line == "READ") {  
        readNext();  
    } else if (line == "COUNT") {  
        displayCount();  
    }  
}
```

```
file.close();  
}
```



```
};
```

```
int main(int argc, char* argv[]) {  
    std::string filename = "commands.txt";  
    if (argc > 1) {  
        filename = argv[1];  
    }  
  
    EmailInboxManager manager;  
    manager.processCommandFile(filename);  
  
    return 0;  
}
```

Claude Code:

/*

*

=====

* CEO EMAIL PRIORITY SYSTEM

*

=====

* Reads a series of commands from a text file and uses a MaxHeap

* (implemented from scratch, list/vector based) to always surface

* the CEO's highest priority unread email.

*

* Supported commands (one per line in the input file):

* EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>

* NEXT

* READ

* COUNT

*

* Priority rules (highest to lowest sender category):

* Boss > Subordinate > Peer > ImportantPerson > OtherPerson

* Within the same category, the newer email (by date) wins.

*

* Design notes:

* - Email : simple data object representing one message.

* - MaxHeap : from-scratch binary max-heap stored in a

* std::vector (a "list"), with insert,

* heapify-up, heapify-down, peek and

* remove(extract-max) all implemented manually.

* No std::priority_queue / std::make_heap used.

* - EmailSystem : the "controller" object that reads commands

* from the file and drives the MaxHeap to

* satisfy each command.

*

=====

*/

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <string>
```

```
#include <vector>
```

```
#include <stdexcept>
```

```
using namespace std;
```

```
//
```

```
=====
```

```
// CLASS: Email
```

```
// Represents a single email message plus the data needed to rank it.
```

```
//
```

```
=====
```

```
class Email {
```

```
public:
```

```
    string senderCategory; // e.g. "Boss", "Peer", ...
```

```
    string subject;        // subject line (may contain spaces)
```

```
    string date;           // original date string, MM-DD-YYYY
```

```
    int   categoryRank;    // numeric priority derived from category
```

```
    long  dateValue;       // numeric YYYYMMDD form, for tie-breaking
```

```
    Email() : senderCategory(""), subject(""), date(""), categoryRank(0),  
dateValue(0) {}
```

```
    Email(const string &category, const string &subj, const string &dateStr) {
```

```
        senderCategory = category;
```

```
        subject = subj;
```

```
date = dateStr;  
categoryRank = rankForCategory(category);  
dateValue = dateToComparableValue(dateStr);  
}
```

```
// Converts a sender category string into a numeric priority.
```

```
// Higher number = higher priority (read first).
```

```
static int rankForCategory(const string &category) {
```

```
    if (category == "Boss")        return 5;
```

```
    if (category == "Subordinate")  return 4;
```

```
    if (category == "Peer")         return 3;
```

```
    if (category == "ImportantPerson") return 2;
```

```
    if (category == "OtherPerson")  return 1;
```

```
    // Unknown categories are treated as lowest priority so the
```

```
    // program never crashes on unexpected input.
```

```
    return 0;
```

```
}
```

```
// Converts "MM-DD-YYYY" into an integer YYYYMMDD so that  
larger
```

```
// numbers represent later (newer) dates and can be compared
```

```
// directly with normal integer comparison.
```

```

static long dateToComparableValue(const string &dateStr) {
    int month = 0, day = 0, year = 0;
    char dash1, dash2;
    istringstream iss(dateStr);
    iss >> month >> dash1 >> day >> dash2 >> year;
    if (iss.fail()) {
        // If the date is malformed, treat it as the oldest possible
        // date so it doesn't accidentally win a tie-break.
        return 0;
    }
    return (static_cast<long>(year) * 10000L) + (month * 100L) + day;
}
};

```

```
//
```

```
=====
```

```
// CLASS: MaxHeap
```

```
// A binary max-heap built from scratch on top of a std::vector
```

```
// (the required "list-based" underlying storage). Emails are ordered
```

```
// first by categoryRank (higher = more important), then by dateValue
```

```
// (newer = more important) when categories tie.
```

```
//
```

```
=====
```

```
class MaxHeap {
```

```
private:
```

```
    vector<Email> heapList; // the underlying list-based storage
```

```
    // ---- index helpers -----
```

```
    int parentIndex(int i) const { return (i - 1) / 2; }
```

```
    int leftChildIndex(int i) const { return (2 * i) + 1; }
```

```
    int rightChildIndex(int i) const { return (2 * i) + 2; }
```

```
    // ---- comparison -----
```

```
    // Returns true if email 'a' should be considered strictly higher
```

```
    // priority than email 'b' (i.e. 'a' belongs above 'b' in the heap).
```

```
    bool isHigherPriority(const Email &a, const Email &b) const {
```

```
        if (a.categoryRank != b.categoryRank) {
```

```
            return a.categoryRank > b.categoryRank;
```

```
        }
```

```
        // Same category: newer date wins.
```

```
        return a.dateValue > b.dateValue;
```

```
    }
```

```
// ---- heapify-up -----  
  
// Restores heap order after an insertion at the end of the list,  
// by repeatedly swapping the new element up while it outranks  
// its parent.
```

```
void heapifyUp(int index) {  
    while (index > 0) {  
        int parent = parentIndex(index);  
        if (isHigherPriority(heapList[index], heapList[parent])) {  
            swap(heapList[index], heapList[parent]);  
            index = parent;  
        } else {  
            break;  
        }  
    }  
}
```

```
// ---- heapify-down -----  
  
// Restores heap order starting from the given index by repeatedly  
// swapping the element down with its highest-priority child until  
// the max-heap property is satisfied. Used after removing the root.  
  
void heapifyDown(int index) {
```



```
int size = static_cast<int>(heapList.size());
while (true) {
    int left = leftChildIndex(index);
    int right = rightChildIndex(index);
    int highest = index;

    if (left < size && isHigherPriority(heapList[left], heapList[highest]))
    {
        highest = left;
    }

    if (right < size && isHigherPriority(heapList[right],
heapList[highest])) {
        highest = right;
    }

    if (highest == index) {
        break; // heap property satisfied
    }

    swap(heapList[index], heapList[highest]);
    index = highest;
}
}
```

public:

```
// ---- insertion -----  
  
// Adds a new email to the end of the list, then heapifies up to  
// restore the max-heap property.  
void insert(const Email &email) {  
    heapList.push_back(email);  
    heapifyUp(static_cast<int>(heapList.size()) - 1);  
}  
  
// ---- peek -----  
  
// Returns the highest-priority email WITHOUT removing it.  
// Caller must check isEmpty() first.  
const Email &peek() const {  
    if (heapList.empty()) {  
        throw runtime_error("peek() called on empty heap");  
    }  
    return heapList[0];  
}  
  
// ---- removal (extract-max) -----  
  
// Removes and returns the highest-priority email.  
Email removeMax() {
```

```
if (heapList.empty()) {  
    throw runtime_error("removeMax() called on empty heap");  
}
```

```
Email top = heapList[0];
```

```
// Move the last element to the root, shrink the list, then
```

```
// heapify down to restore order.
```

```
heapList[0] = heapList.back();
```

```
heapList.pop_back();
```

```
if (!heapList.empty()) {
```

```
    heapifyDown(0);
```

```
}
```

```
return top;
```

```
}
```

```
bool isEmpty() const {
```

```
    return heapList.empty();
```

```
}
```

```
int size() const {
```

```
    return static_cast<int>(heapList.size());
```

```

    }

};

//
=====

// CLASS: EmailSystem

// The controller object. Reads commands from a file and uses the
// MaxHeap to service them.

//
=====

class EmailSystem {
private:
    MaxHeap inbox; // the priority queue of unread emails

    // Trims leading/trailing whitespace from a string.
    static string trim(const string &s) {
        size_t start = s.find_first_not_of(" \t\r\n");
        if (start == string::npos) return "";
        size_t end = s.find_last_not_of(" \t\r\n");
        return s.substr(start, end - start + 1);
    }
}

```

```
// Splits "category,subject,date" into its 3 comma-separated parts.
```

```
// Subject may contain spaces (but not commas, per spec).
```

```
static vector<string> splitByComma(const string &line) {
```

```
    vector<string> parts;
```

```
    stringstream ss(line);
```

```
    string field;
```

```
    while (getline(ss, field, ',')) {
```

```
        parts.push_back(trim(field));
```

```
    }
```

```
    return parts;
```

```
}
```

```
// ---- command handlers -----
```

```
// Handles: EMAIL <category>,<subject>,<date>
```

```
void handleEmailCommand(const string &args) {
```

```
    vector<string> parts = splitByComma(args);
```

```
    if (parts.size() != 3) {
```

```
        cout << "Malformed EMAIL command, skipping: " << args << endl;
```

```
        return;
```

```
    }
```

```

    Email newEmail(parts[0], parts[1], parts[2]);
    inbox.insert(newEmail);
}

// Handles: NEXT -- show highest priority email without removing it
void handleNextCommand() {
    if (inbox.isEmpty()) {
        cout << "No unread emails." << endl;
        return;
    }
    const Email &top = inbox.peek();
    cout << "Sender: " << top.senderCategory << endl;
    cout << "Subject: " << top.subject << endl;
    cout << "Date: " << top.date << endl;
}

// Handles: READ -- remove/consume the highest priority email
void handleReadCommand() {
    if (inbox.isEmpty()) {
        cout << "No emails to read." << endl;
        return;
    }

```

```

Email handled = inbox.removeMax();

cout << "Read and handled email from " << handled.senderCategory
    << " - \"" << handled.subject << "\"" << endl;

}

```

```

// Handles: COUNT -- report number of unread emails

void handleCountCommand() {
    cout << "Unread email count: " << inbox.size() << endl;
}

```

public:

```

// Reads the given file line by line and dispatches each command.

void run(const string &filename) {
    ifstream infile(filename.c_str());
    if (!infile.is_open()) {
        cerr << "Error: could not open command file: " << filename << endl;
        return;
    }
}

```

```

string line;

while (getline(infile, line)) {
    string trimmedLine = trim(line);
}

```

```
if (trimmedLine.empty()) continue; // skip blank lines
```

```
if (trimmedLine.rfind("EMAIL", 0) == 0) {
```

```
    // Everything after "EMAIL" (and one space) is the payload.
```

```
    string args = trimmedLine.substr(5);
```

```
    args = trim(args);
```

```
    cout << "> EMAIL " << args << endl;
```

```
    handleEmailCommand(args);
```

```
} else if (trimmedLine == "NEXT") {
```

```
    cout << "> NEXT" << endl;
```

```
    handleNextCommand();
```

```
} else if (trimmedLine == "READ") {
```

```
    cout << "> READ" << endl;
```

```
    handleReadCommand();
```

```
} else if (trimmedLine == "COUNT") {
```

```
    cout << "> COUNT" << endl;
```

```
    handleCountCommand();
```

```
} else {
```

```
    cout << "Unknown command, skipping: " << trimmedLine <<
```

```
endl;
```

```
}
```

```
cout << "-----" << endl;
```



```
}
```

```
infile.close();
```

```
}
```

```
};
```

```
//
```

```
=====
```

```
// main
```

```
// Reads the command filename from the command line if given, otherwise
```

```
// defaults to "commands.txt" in the current directory.
```

```
//
```

```
=====
```

```
int main(int argc, char *argv[]) {
```

```
    string filename = "commands.txt";
```

```
    if (argc > 1) {
```

```
        filename = argv[1];
```

```
    }
```

```
EmailSystem system;
```

```
system.run(filename);
```

```
return 0;
```

```
}
```

5. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Answer:

Correctness: Both programs actually implement the same strategy almost for the required email priority system. Gemini uses the node based Maxheap whereas Claude uses vector-based heap. Gemini’s potential issue from creating an Email with an empty date can cause issue later.

Execution Time: Both GenAI generated code uses heapify operations, so insertion are generally $O(\log N)$, Claude’s Vector implementation has less pointer traversal overhead, while Gemini must locate for nodes in its tree.

Space Complexity: Both Claude and Gemini Code requires $O(n)$ space for n Email. Gemini uses some extra memory because of parents, left, and right pointers.

Maintainability: Gemini and Claude both code uses classes and separate responsibilities very clearly. Gemini’s code some structures are like very straight forward for example, Email, Maxheap, and EmailInboxManager. While the Claude’s generated code includes additional parsing and handling code.

6. Select one to use as the basis for this assignment and justify why you picked it.

Answer: I would choose Gemini's Code as the basis for this assignment. The reason for choosing Gemini's generated code, its node based MaxHeap more clearly follows the required list-based implementation. The Gemini generated code is also well organized into separate Maxheap, Email, and EmailInboxManager classes which makes it easier to understand and maintain in future.

7. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Answer:

Correctness: I will fix the empty-date Email issue in Gemini's generated code because that could cause the program to crash and I would also adjust the output to match the required format of assignment.

Execution Time: I would keep the Gemini's generated code Maxheap operation efficient, with insertion and removal using $O(\log N)$ heap operation

Space Complexity: I would keep the $O(n)$ space requirement as it was while using node-based Maxheap required by the assignment.

Maintainability: For the Gemini's generated code I will try to improve the comments little bit by giving some extra explanation, simplifies some part of code, and also makes the command handling and output easier to understand.